

IA-REFLEXION-C: Descubrimos que Lomuto pone el pivote en su lugar final aislando menores a la izquierda y mayores a la derecha.
Sin embargo, si los datos son todos iguales o ya están casi ordenados (eligiendo el último como pivote),
la partición no divide el problema por la mitad, volviendo al algoritmo ineficiente con un tiempo de $O(n^2)$.

"""

Semana 10 — Quick Sort y Veredicto StreamMX

Estructura de Datos Básica · UAN · LSC

Dr. Eligardo Cruz Sánchez

Equipo: [nombres del equipo]

Fecha: [fecha de la sesión]

"""

```
import time
import random
import copy
```

#

EVIDENCIAS DE COMPRENSIÓN (llenar ANTES de codificar)

#

COMPRENDE-1: ¿Cuál es el invariante del pivote en Lomuto?

En cualquier momento de la iteración, los elementos desde el índice inicial

hasta el puntero 'i' son menores o iguales al pivote, y los elementos desde

'i + 1' hasta 'j - 1' son estrictamente mayores al pivote.

COMPRENDE-2: ¿Cuándo y por qué Quick Sort degenera a $O(n^2)$?

Degenera cuando el arreglo ya está ordenado (ascendente o descendente) o cuando

todos los elementos son iguales. En estos casos, Lomuto elige el extremo como pivote,

resultando en particiones extremadamente desbalanceadas de tamaño 0 y n-1 en cada llamada.

COMPRENDE-3: ¿Por qué Quick Sort es inestable y Merge Sort estable?

Quick Sort es inestable porque realiza intercambios de elementos a larga distancia,

saltando sobre otros elementos y destruyendo el orden relativo original de llaves idénticas.

Merge Sort es estable porque, al fusionar mitades, utiliza el operador '<=' para

darle prioridad incondicional al subarreglo izquierdo en caso de empate.

ESTRATEGIA: Describe brevemente el plan del equipo para el benchmark

Incrementaremos temporalmente el límite de recursión del intérprete de Python.

```
# Esto nos permitirá evitar un "Stack Overflow" prematuro y lograr diferenciar
# analíticamente entre un fallo por agotamiento de RAM (complejidad espacial en la pila)
# y una degradación severa de rendimiento (complejidad temporal  $O(n^2)$ ).
```

```
#
```

```
# SECCIÓN 1 — QUICK SORT (partición Lomuto)
#
```

```
def particionar_lomuto(A, izq, der, key=lambda x: x):
```

```
    """
```

```
    Particiona A[izq..der] usando Lomuto.
```

```
    El pivote es A[der] (último elemento).
```

```
    Retorna el índice final del pivote.
```

```
    """
```

```
    # DECISIÓN: Elegimos el último elemento por la simplicidad heurística de la
```

```
    # implementación base de Lomuto. Aunque nos expone al peor caso asintótico en
```

```
    # arreglos ya ordenados, reduce la sobrecarga computacional en particiones
```

```
    # pequeñas al no requerir cálculos adicionales para encontrar la mediana.
```

```
    pivote = key(A[der])
```

```
    i = izq - 1
```

```
    for j in range(izq, der):
```

```
        # Si A[j] <= pivote, incrementar i e intercambiar A[i] con A[j]
```

```
        if key(A[j]) <= pivote:
```

```
            i += 1
```

```
            A[i], A[j] = A[j], A[i]
```

```
    # Colocar el pivote en su posición definitiva (intercambiar A[i+1] con A[der])
```

```
    A[i + 1], A[der] = A[der], A[i + 1]
```

```
    # Retornar el índice del pivote
```

```
    return i + 1
```

```
def quick_sort(A, izq=None, der=None, key=lambda x: x):
```

```
    """
```

```
    Quick Sort recursivo sobre A[izq..der].
```

```
    Llama a particionar_lomuto para obtener el índice del pivote.
```

```
    """
```

```
    if izq is None:
```

```
        izq = 0
```

```
    if der is None:
```

```

    der = len(A) - 1

    if izq < der:
        # Obtener el índice del pivote usando particionar_lomuto
        p = particionar_lomuto(A, izq, der, key)

        # Llamar recursivamente sobre A[izq..p-1] y A[p+1..der]
        quick_sort(A, izq, p - 1, key)
        quick_sort(A, p + 1, der, key)

#


---




---


# SECCIÓN 2 — EXPERIMENTO DE ESTABILIDAD
#


---




---



# Datos simulados StreamMX: tuplas (puntaje, titulo, fecha_subida)
datos_streamx = [
    (7.5, "El Laberinto", "lunes"),
    (8.2, "Interestelar", "lunes"),
    (7.5, "La Señal", "martes"),
    (9.1, "Origen", "lunes"),
    (7.5, "Gravedad", "miércoles"),
    (8.2, "Tenet", "martes"),
    (7.5, "Dune", "jueves"),
]

def verificar_estabilidad(datos_originales, datos_ordenados, campo_clave=0):
    """
    Verifica si el ordenamiento preservó el orden relativo de elementos
    con el mismo valor en campo_clave.
    Retorna (True, "") si es estable, (False, detalle) si no.
    """
    from collections import defaultdict

    orden_original = defaultdict(list)
    for idx, elemento in enumerate(datos_originales):
        clave = elemento[campo_clave] if isinstance(elemento, (list, tuple)) else elemento
        orden_original[clave].append(idx)

    posicion_en_original = defaultdict(int)
    for elemento in datos_ordenados:
        clave = elemento[campo_clave] if isinstance(elemento, (list, tuple)) else elemento
        grupo = orden_original[clave]
        ptr = posicion_en_original[clave]

```

```

if ptr >= len(grupo):
    return False, f"Clave {clave}: más ocurrencias en ordenado que en original"
    posicion_en_original[clave] += 1

posicion_en_original = defaultdict(int)
ultimo_idx = defaultdict(lambda: -1)
for elemento in datos_ordenados:
    clave = elemento[campo_clave] if isinstance(elemento, (list, tuple)) else elemento
    ptr = posicion_en_original[clave]
    idx_original = orden_original[clave][ptr]
    if idx_original <= ultimo_idx[clave]:
        detalle = (f"Inestabilidad detectada en clave '{clave}': "
                    f"elemento con índice original {idx_original} aparece "
                    f"después de elemento con índice {ultimo_idx[clave]}")
        return False, detalle
    ultimo_idx[clave] = idx_original
    posicion_en_original[clave] += 1

return True, "Ordenamiento estable: orden relativo preservado en todos los grupos"

```

#

SECCIÓN 3 — BENCHMARK COMPARATIVO (MS de S9 vs QS de S10)

#

```

def generar_distribucion(n, tipo):
    if tipo == 'aleatorio':
        return [random.randint(1, n) for _ in range(n)]
    elif tipo == 'ordenado':
        return list(range(n))
    elif tipo == 'inverso':
        return list(range(n, 0, -1))
    elif tipo == 'duplicados':
        base = [7] * int(n * 0.8)
        ruido = [random.randint(1, n) for _ in range(n - len(base))]
        arreglo = base + ruido
        random.shuffle(arreglo)
        return arreglo
    else:
        raise ValueError(f"Distribución desconocida: {tipo}")

def medir_tiempo(funcion, arreglo):
    copia = copy.deepcopy(arreglo)

```

```

inicio = time.perf_counter()
funcion(copia)
fin = time.perf_counter()
return fin - inicio

```

```

def merge_sort(A, key=lambda x: x):
    """
    Implementación completa de Merge Sort de S9 adaptada para extraer llaves.
    """
    if len(A) <= 1:
        return A

    mid = len(A) // 2
    izq = merge_sort(A[:mid], key)
    der = merge_sort(A[mid:], key)

    resultado = []
    i = j = 0
    while i < len(izq) and j < len(der):
        if key(izq[i]) <= key(der[j]): # Condición fundamental para la estabilidad
            resultado.append(izq[i])
            i += 1
        else:
            resultado.append(der[j])
            j += 1

    resultado.extend(izq[i:])
    resultado.extend(der[j:])

    # Para emular el comportamiento in-place en pruebas donde sea requerido
    A[:] = resultado
    return A

```

```

def ejecutar_benchmark():
    import sys
    sys.setrecursionlimit(110_000)

    tamanos = [1_000, 10_000, 100_000]
    distribuciones = ['aleatorio', 'ordenado', 'inverso', 'duplicados']

    print("=" * 85)
    print(f'{"n":>10} | {"Distribución":>12} | {"Merge Sort (s)":>18} | {"Quick Sort (s)":>18}')
    print("=" * 85)

    for n in tamanos:
        for dist in distribuciones:

```

```

A = generar_distribucion(n, dist)

t_ms = medir_tiempo(merge_sort, A)

try:
    t_qs = medir_tiempo(quick_sort, A)
    resultado_qs = f"{t_qs:>18.4f}"
except RecursionError:
    resultado_qs = f"{'CRASH (Stack Overflow)':>18}"

print(f"{n:>10} | {dist:>12} | {t_ms:>18.4f} | {resultado_qs}")
print("-" * 85)

```

REFLEXIONA-1: ¿En qué distribución Quick Sort fue MÁS lento o crasheó?
 # Quick Sort crasheó sistemáticamente en las distribuciones 'ordenado', 'inverso'
 # y 'duplicados' en el tamaño de 100,000 registros, confirmando la vulnerabilidad
 # de particionar asimétricamente un arreglo degenerado.

REFLEXIONA-2: ¿Qué significa "CRASH (Stack Overflow)" en términos de espacio $O(n)$?
 # Significa que el sistema agotó la memoria física asignada al proceso. En el peor
 # caso asintótico, Lomuto degrada la profundidad del árbol de recursión de $O(\log n)$
 # a $O(n)$. Para $n=100,000$, el intérprete abre 100,000 marcos de pila sucesivos en la RAM
 # antes de retornar, reventando el límite configurado del lenguaje.

REFLEXIONA-3: Con estos datos, ¿qué recomendarías a la Ing. Sofía para cada distribución?
 # Recomendaríamos Merge Sort en escenarios con fuerte nivel de pre-ordenamiento o
 alta
 # tasa de empatía de llaves. Quick Sort se recomendaría sólo en entornos de memoria
 RAM
 # limitada con ingesta de datos puramente aleatorios.

```

if __name__ == "__main__":
    # Sección 1: Verificar que Quick Sort ordena correctamente
    prueba = [9, 3, 7, 1, 5, 8, 2, 4, 6]
    quick_sort(prueba)
    print("Prueba básica:", prueba)
    assert prueba == sorted([9, 3, 7, 1, 5, 8, 2, 4, 6]), "Quick Sort falla en prueba básica"
    print("---")

    # Sección 2: Experimento de estabilidad
    # Copia ordenada con merge_sort evaluando solo el índice 0 (puntaje)
    copia_ms = copy.deepcopy(datos_streamx)
    merge_sort(copia_ms, key=lambda x: x[0])

    # Copia ordenada con quick_sort evaluando solo el índice 0

```

```

copia_qs = copy.deepcopy(datos_streamx)
quick_sort(copia_qs, key=lambda x: x[0])

# Verificar y mostrar resultados
exito_ms, detalle_ms = verificar_estabilidad(datos_streamx, copia_ms, campo_clave=0)
print(f"Merge Sort: {detalle_ms}")

exito_qs, detalle_qs = verificar_estabilidad(datos_streamx, copia_qs, campo_clave=0)
print(f"Quick Sort: {detalle_qs}")

print("\nVisualización de registros con puntaje 7.5:")
print("Merge Sort:", [p for p in copia_ms if p[0] == 7.5])
print("Quick Sort:", [p for p in copia_qs if p[0] == 7.5])
print("\nIniciando Benchmark...")
print("---")

# Sección 3: Benchmark
ejecutar_benchmark()

```

#

```

# VALIDA: VEREDICTO STREAMMX — EQUIPO [NOMBRE]
#
# DISTRIBUCIÓN A — Rankings casi ordenados (pocos cambios del día anterior):
# Algoritmo recomendado: Merge Sort
# Razón tiempo:    Garantiza siempre un comportamiento de orden  $O(n \log n)$ . Quick
Sort colapsa a  $O(n^2)$ .
# Razón espacio:   Su necesidad auxiliar  $O(n)$  previene el Stack Overflow fatal que
sufre QS en este escenario.
# Razón estabilidad: Al ser una actualización de un ranking previo, preservar la
antigüedad en empates es crucial.
# Razón duplicados: Merge sort no penaliza su división heurística ante clústeres de
puntajes similares.
#
# DISTRIBUCIÓN B — Datos frescos aleatorios (flujo del día):
# Algoritmo recomendado: Quick Sort
# Razón tiempo:    Muestra ventajas asintóticas constantes superiores debido a la
localidad de su acceso en caché ( $O(n \log n)$  rápido).
# Razón espacio:   Al estar bien distribuido, opera in-place con una pila de recursión
mínima  $O(\log n)$ .
# Razón estabilidad: Irrelevante, dado que es flujo nuevo sin un orden inherente que
proteger.
#
# DISTRIBUCIÓN C — Millones de empates en puntaje (caso crítico):
# Algoritmo recomendado: Merge Sort
# Razón tiempo:    Conserva el rendimiento  $O(n \log n)$  mientras el catálogo crece
exponencialmente.

```

Razón espacio: Aunque gasta RAM extra, es preferible a agotar recursivamente la pila principal del procesador.

Razón duplicados: CRÍTICO: Lomuto puro frente a millones de empates realiza iteraciones nulas,

fractura el arreglo en proporciones extremas y desencadena la muerte del hilo por Stack Overflow.

#

CONCLUSIÓN GENERAL:

Recomendamos implementar Merge Sort como solución estructural para StreamMX, debido a su resiliencia matemática

y su propiedad intrínseca de estabilidad; no obstante, si los costos en la nube por RAM se vuelven insostenibles,

se deberá migrar a un Quick Sort avanzado (usando partición de 3 bandas de Dijkstra y pivote aleatorio) perdiendo la estabilidad.

#

IA-REFLEXION-A: La IA aclaró el propósito conceptual de los punteros en Lomuto ('i' como frontera de menores y 'j' como explorador).

Con esta base teórica, el equipo logró implementar sin ayuda la lógica de intercambios en el ciclo y la reubicación final del pivote.

Asimismo, codificaron independientemente la estructura recursiva de Quick Sort y la ejecución completa del benchmark.

IA-REFLEXION-R: El ítem de mayor dificultad fue la Pregunta D1, específicamente el rastreo del puntero 'i' en la traza de Lomuto cuando no ocurren intercambios.

El equipo falló inicialmente al no considerar el valor estricto de inicialización (izq - 1) previo a la entrada del ciclo iterativo.

Antes de la defensa real, necesitan repasar rigurosamente la ejecución de pruebas de escritorio sobre casos borde, monitoreando los estados de memoria iniciales.

IA-REFLEXION-V: Sí, los casos detectaron violaciones críticas a los invariantes de memoria y lógica del equipo.

El Caso A (Fuga de Límite Inferior) fue el más revelador, ya que demostró que estandarizar 'i = -1' corrompe las ramas derechas.

Esto evidenció que el equipo aún no dominaba la gestión de desplazamientos relativos en subarreglos durante la recursión in-place.

IA-REFLEXION-P: Al equipo le queda pendiente descubrir cómo gestionar los punteros 'izq' y 'der'

en el código de Búsqueda Binaria para calcular la mitad exacta sin provocar desbordamientos,

y cómo evitar ciclos infinitos cuando la película buscada no existe en el catálogo.